

Week10_Ch06. 자연어 처리-임베딩(Word2vec ~)

6.5. Word2Vec

6.5.1. Word2Vec 개념

- 2013년 구글에서 공개한 임베딩 모델
- 단어 간의 유사성을 측정하기 위해 분포 가설을 기반으로 개발되었음

분포 가설(distributional hypothesis)

- 같은 문맥에서 함께 자주 나타나는 단어들은 서로 유사한 의미를 가질 가능성이 높다는 가정
- 동시 발생(co-occurrence) 확률 분포를 이용해 단어 간의 유사성을 측정
- 예시) '내일 자동차를 타고 부산에 간다'와 '내일 비행기를 타고 부산에 간다'
⇒ '자동차'와 '비행기'는 주변에 분포한 단어들이 동일하거나 유사
⇒ 비슷한 의미를 가질 것이라고 예상
- 이러한 가정을 통해 **분산 표현**을 학습

분산 표현(distributed representation)

- 단어를 고차원 벡터 공간에 매핑하여 단어의 의미를 담는 것
- 분포 가설에 따라 단어의 의미는 문맥상 분포적 특성을 통해 나타남
⇒ 유사한 문맥에서 등장하는 단어는 비슷한 벡터 공간상 위치를 갖게 됨
⇒ '자동차'와 '비행기'는 벡터 공간에서 서로 가까운 위치에 표현
- 효과
 - 빈도 기반의 벡터화 기법에서 발생했던 단어의 의미 정보를 저장하지 못하는 한계를 극복
 - 대량의 텍스트 데이터에서 단어 간의 관계를 파악
 - 벡터 공간상에서 유사한 단어를 군집화해 단어의 의미 정보를 효과적으로 표현

이점

- 다양한 자연어 처리 작업에서 높은 성능을 보임
- 다운스트림 작업에서 더 뛰어난 성능을 보임

6.5.2. 단어 벡터화

단어 벡터화 방법

- **희소 표현(sparse representation)**

소	0	1	0	0	0
읽고	1	0	0	0	0
외양간	0	0	0	1	0
고친다	0	0	0	0	1

- 빈도 기반 방법
- 원-핫 인코딩, TF-IDF

- **밀집 표현(dense representation)**

소	0.3914	-0.1749	...	0.5912	0.1321
읽고	-0.2893	0.3814	...	-0.1492	-0.2814

외양간	0.4812	0.1214	...	-0.2745	0.0132
고친다	-0.1314	-0.2809	...	0.2014	0.3016

- Word2Vec

희소 표현(sparse representation)

- 대부분의 벡터 요소가 0으로 표현되는 희소 표현 방법
- 예시
 - 모델의 단어 사전이 5,000개의 단어로 이루어졌다면, 10개의 토큰으로 이루어진 입력 텍스트를 원-핫 인코딩으로 표현하면 최소한 4,990개의 0이 포함된 벡터로 표현됨
- 단점
 - 단어 사전의 크기가 커지면 벡터의 크기도 커지므로 공간적 낭비가 발생
 - 단어 간의 유사성을 반영하지 못함
 - 벡터 간의 유사성을 계산하는 데 많은 비용이 발생

밀집 표현(dense representation)

- 단어를 고정된 크기의 실수 벡터로 표현
- 단어 사전의 크기가 커지더라도 벡터의 크기가 커지지 않음
- 벡터 공간상에서 단어 간의 거리를 효과적으로 계산할 수 있음
- 벡터의 대부분이 0이 아닌 실수로 이루어져 있어 효율적으로 공간을 활용할 수 있음
- 학습을 통해 단어를 벡터화
 - ⇒ 단어의 의미를 비교할 수 있음
- 단어 임베딩 벡터
 - : 밀집 표현된 벡터
 - ⇒ Word2Vec은 대표적인 단어 임베딩 기법 중 하나
- Word2Vec
 - 밀집 표현을 위해 **CBoW, Skip-gram**이라는 두 가지 방법 사용

6.5.3. CBoW(Continuous Bag of Words)

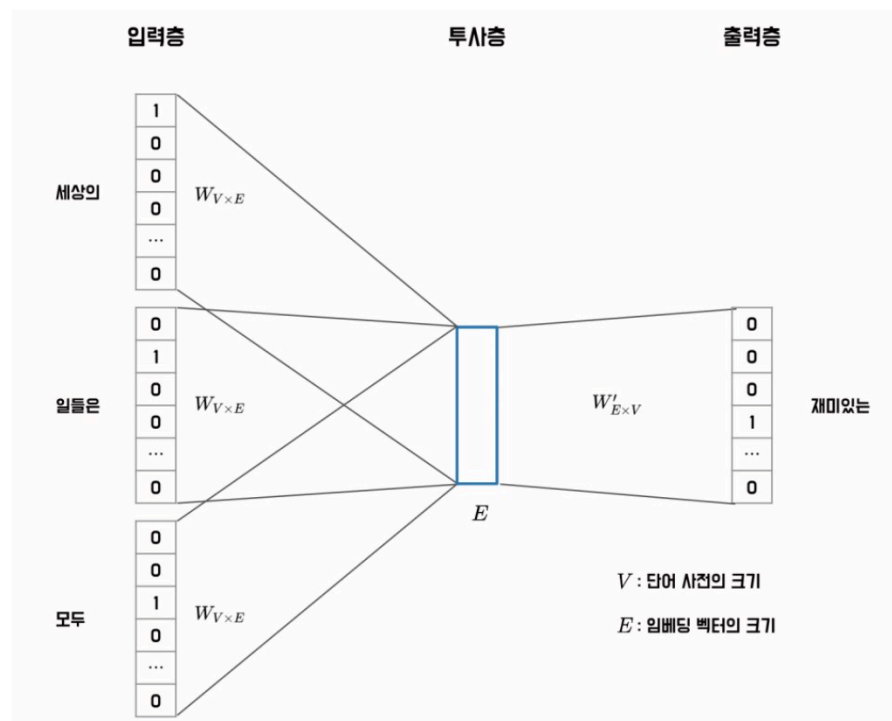
CBoW 개념

- CBoW
 - 주변에 있는 단어를 가지고 중간에 있는 단어를 예측하는 방법
- 중심 단어(Center Word)
 - 예측해야 할 단어
- 주변 단어(Context Word)
 - 예측에 사용되는 단어들
- 윈도우(Window)
 - 중심 단어를 맞추기 위해 몇 개의 주변 단어를 고려할지 정한 범위
 - 윈도우를 활용해 주어진 하나의 문장에서 첫 번째 단어부터 중심 단어로 하여 마지막 단어까지 학습
- 슬라이딩 윈도우(Sliding Window)
 - 학습을 위해 윈도우를 이동해 가며 학습하는 방법
 - 슬라이딩 윈도우를 사용해 한 번의 학습으로 여러 개의 중심 단어와 그에 대한 주변 단어를 학습
- 학습 데이터
 - 예시) 하나의 입력 문장에서 윈도우 크기가 2일 때 학습 데이터의 구성

입력 문장	학습 데이터
(세상의 재미있는 일들은)모두 밤에 일어난다	(재미있는, 일들은 세상의)
(세상의 재미있는 일들은 모두)밤에 일어난다	(세상의, 일들은, 모두 재미있는)
(세상의 재미있는 일들은 모두 밤에)일어난다	(세상의, 재미있는, 모두, 밤에 일들은)
세상의(재미있는 일들은 모두 밤에 일어난다)	(재미있는, 일들은, 밤에, 일어난다 모두)
세상의 재미있는(일들은 모두 밤에 일어난다)	(일들은, 모두, 일어난다 밤에)
세상의 재미있는 일들은(모두 밤에 일어난다)	(모두, 밤에 일어난다)

- (주변 단어 | 중심 단어)로 구성
⇒ 대량의 말뭉치에서 효율적으로 단어의 분산 표현을 학습할 수 있음
- 얻어진 학습 데이터는 인공 신경망을 학습하는 데 사용

CBow 모델 구조



- 각 입력 단어의 원-핫 벡터를 입력값으로 받음 → 입력 문장 내 모든 단어의 임베딩 벡터를 평균 내어 중심 단어의 임베딩 벡터를 예측
 1. 입력 단어는 원-핫 벡터로 표현
 2. 투사층(Projection Layer)에 입력
 - 투사층: 원-핫 벡터의 인덱스에 해당하는 임베딩 벡터를 반환하는 순람표(Lookup table, LUT) 구조
 3. 각 단어는 E 크기의 임베딩 벡터로 변환
 4. 단어의 임베딩 벡터의 평균값을 계산
 5. 계산된 평균 벡터를 가중치 행렬 $W'_{E \times V}$ 과 곱하면 V 크기의 벡터를 얻음
 6. 이 벡터에 소프트맥스 함수를 이용해 중심 단어를 예측

6.5.4. Skip-gram

Skip-gram 개념

- CBoW와 반대로 중심 단어를 입력으로 받아서 주변 단어를 예측하는 모델
- 중심 단어를 기준으로 양쪽으로 윈도우 크기만큼의 단어들을 주변 단어로 삼아 훈련 데이터셋을 만듦
- 학습 데이터
 - 예시) 윈도우 크기가 2일 때 학습 데이터 구성

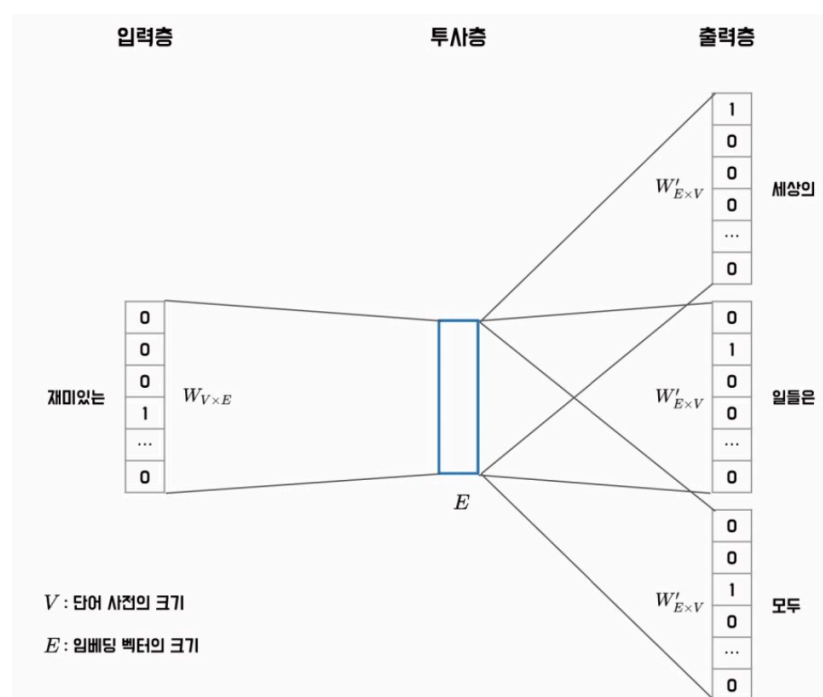
입력 문장	학습 데이터
(세상의 재미있는 일들은)모두 밤에 일어난다	(세상의 재미있는), (세상의 일들은)
(세상의 재미있는 일들은 모두)밤에 일어난다	(재미있는 세상의), (재미있는 일들은), (재미있는 모두)
(세상의 재미있는 일들은 모두 밤에)일어난다	(일들은 세상의), (일들은 재미있는), (일들은 모두), (일들은 밤에)
세상의(재미있는 일들은 모두 밤에 일어난다)	(모두 재미있는), (모두 일들은), (모두 밤에), (모두 일어난다)
세상의 재미있는(일들은 모두 밤에 일어난다)	(밤에 일들은), (밤에 모두), (밤에 일어난다)
세상의 재미있는 일들은(모두 밤에 일어난다)	(일어난다 모두), (일어난다 밤에)

- CBoW는 하나의 윈도우에서 하나의 학습 데이터가 만들어지는 반면, Skip-gram은 중심 단어와 주변 단어를 하나의 쌍으로 하여 여러 학습 데이터가 만들어짐
- ⇒ Skip-gram은 하나의 중심 단어를 통해 여러 개의 주변 단어를 예측
- ⇒ 더 많은 학습 데이터셋을 추출
- ⇒ 일반적으로 CBoW보다 더 뛰어난 성능을 보임

장점

- 하나의 중심 단어를 통해 여러 개의 주변 단어를 예측하여 더 많은 학습 데이터셋을 추출하여 성능이 뛰어남
- 비교적 드물게 등장하는 단어를 더 잘 학습할 수 있게 됨
- 단어 벡터 공간에서 더 유의미한 거리 관계를 형성할 수 있음

Skip-gram 모델 구조



1. 입력 단어의 원-핫 벡터를 투사층에 입력하여 해당 단어의 임베딩 벡터를 가져옴
2. 입력 단어의 임베딩과 $W'_{E \times V}$ 가중치와의 곱셈을 통해 V 크기의 벡터를 얻음
3. 이 벡터에 소프트맥스 연산을 취함으로써 주변 단어를 예측
 - 소프트맥스 연산은 모든 단어를 대상으로 내적 연산을 수행

단점

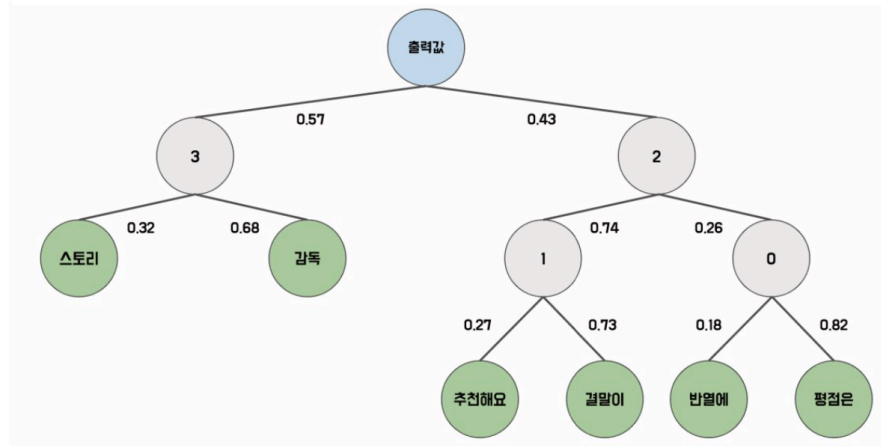
- 말뭉치의 크기가 커지면 단어 사전의 크기도 커지므로 대량의 말뭉치를 통해 Word2Vec 모델을 학습할 때 학습 속도가 느려짐
- ⇒ 이를 해결하기 위해 **계층적 소프트맥스**와 **네거티브 샘플링 기법**을 적용해 학습 속도가 느려지는 문제를 완화

6.5.5. 계층적 소프트맥스(Hierarchical Softmax)

계층적 소프트맥스 개념

- 출력층을 이진 트리 구조로 표현해 연산을 수행
 - 자주 등장하는 단어일수록 트리의 상위 노드에 위치
 - 드물게 등장하는 단어일수록 하위 노드에 배치
- ⇒ 일반적인 소프트맥스 연산에 비해 빠른 속도와 효율성을 보임

계층적 소프트맥스 구조



- 각 노드: 학습이 가능한 벡터를 가짐
- 입력값은 해당 노드의 벡터와 내적값을 계산한 후 시그모이드 함수를 통해 확률을 계산
- 앞 노드(Leaf Node)
 - 가장 깊은 노드
 - 각 단어를 의미
- 모델은 각 노드의 벡터를 최적화하여 단어를 잘 예측할 수 있게 함
- 각 단어의 확률은 경로 노드의 확률을 곱해서 구함
 - 예시) '추천해요' 단어는 $0.43 \times 0.74 \times 0.27 = 0.085914$ 의 확률을 갖게됨
 - ⇒ 1, 2번 노드의 벡터만 최적화

연산

- 단어 사전의 크기: V
- 일반적인 소프트맥스 연산: $O(V)$ 의 시간 복잡도
- 계층적 소프트맥스 연산: $O(\log_2 V)$ 의 시간 복잡도

6.5.6. 네거티브 샘플링(Negative Sampling)

네거티브 샘플링 개념

- Word2Vec 모델에서 사용되는 확률적인 샘플링 기법
- 전체 단어 집합에서 일부 단어를 샘플링하여 오답 단어로 사용
- 학습 윈도 내에 등장하지 않는 단어를 n 개 추출 → 정답 단어와 함께 소프트맥스 연산 수행
 - ⇒ 전체 단어의 확률을 계산할 필요 없이 모델을 효율적으로 학습

추출 확률 계산

- 학습 윈도 내에 등장하지 않는 추출할 단어 n : 5~20개 사용
- 각 단어가 추출될 확률

$$P(w_i) = \frac{f(w_i)^{0.75}}{\sum_{j=0}^V f(w_j)^{0.75}}$$

- $f(w_i)$
 - 각 단어 w_i 의 출현 빈도수
 - 말뭉치 내에 '추천해요'가 100번 등장했고, 전체 단어의 빈도가 2000이라면 $f(\text{추천해요}) = \frac{100}{2000} = 0.05$
 - 단어 w_i 가 네거티브 샘플로 추출될 확률
- 출현 빈도수에 0.75 제곱한 값을 정규화 상수로 사용
 - 실험을 통해 얻어진 최적의 값

모델 학습

- 입력 단어 쌍이 데이터로부터 추출된 단어 쌍인지, 네거티브 샘플링으로 생성된 단어 쌍인지 이진 분류
⇒ 로지스틱 회귀 모델 사용
- 학습 과정에서는 추출한 단어의 확률 분포를 구하기 위해 먼저 각 단어에 대한 가중치를 학습

Skip-gram 모델과 Word2Vec 모델의 훈련 데이터 비교

입력 데이터	출력 데이터
재미있는	세상의
재미있는	일들은
재미있는	모두

→

입력 데이터	출력 데이터
재미있는, 세상의	1
재미있는, 일들은	1
재미있는, 모두	1
재미있는, 걸어가다	0
재미있는, 주십시오	0
재미있는, 미리	0
재미있는, 안녕하세요	0

- 네거티브 샘플링 모델의 훈련 데이터
 - '재미있는'이라는 중심 단어를 통해 세 쌍의 학습 데이터를 추출
 - 네거티브 샘플링 Word2Vec 모델은 실제 데이터에서 추출된 단어 쌍은 1로, 네거티브 샘플링을 통해 추출된 가짜 단어쌍은 0으로 레이블링
⇒ 다중 분류에서 이진 분류로 학습 목적이 바뀌게 됨

일반적인 Word2Vec 모델과 네거티브 샘플링 모델의 출력 구조의 차이



- 네거티브 샘플링 모델
 - 입력 단어의 임베딩과 해당 단어가 맞는지 여부를 나타내는 레이블(1/0)을 가져와 내적 연산을 수행
 - 내적 연산을 통해 얻은 값은 시그모이드 함수를 통해 확률값으로 변환
 - 레이블이 1인 경우 해당 확률값이 높아지도록, 레이블이 0인 경우 해당 확률값이 낮아지도록 모델의 가중치가 최적화됨

6.5.7. 모델 실습: Skip-gram

Week11_예습과제_방민지.ipynb 참조

6.5.8. 모델 실습: Gensim

Week11_예습과제_방민지.ipynb 참조

6.5.9. Word2Vec 단점

- 분포 가설을 통해 빠르게 단어의 임베딩을 학습할 수 있지만 단어의 형태학적 특징을 반영하지 못한다는 한계가 있음
 - 예시) 교착어로서 한국어는 어근, 접사, 조사 등으로 이루어지는 규칙을 가지고 있음
⇒ Word2Vec 모델에서는 이러한 구조적 특징을 제대로 학습하기가 어려움
 - 이러한 한계는 제한된 단어 사전에서 많은 OOV를 발생시키는 원인이 됨

6.6. fastText

6.6.1. fastText 개념

- 2015년 메타의 FAIR 연구소에서 개발한 오픈소스 임베딩 모델
- 텍스트 분류, 텍스트 마이닝을 위한 알고리즘

동작 방식

- 단어와 문장을 벡터로 변환하는 기술을 기반으로 함
 - ⇒ 머신러닝 알고리즘이 텍스트 데이터를 분석하고 이해하는 데 사용됨
- 벡터화 기술은 Word2Vec과 유사하지만, fastText는 단어의 하위 단어를 고려함
 - ⇒ 더 높은 정확도와 성능 제공
- 하위 단어를 고려하기 위해 N-gram을 사용해 단어를 분해하고 벡터화

벡터화

- 단어의 벡터화를 위해 <,>와 같은 특수 기호를 사용하여 단어의 시작과 끝을 나타냄
 - ⇒ 이러한 기호는 단어의 하위 문자열을 고려하는 데 중요한 역할을 함
 - 기호가 추가된 단어는 N-gram을 사용하여 **하위 단어 집합(Subword set)**으로 분해됨
 - 예시) '서울특별시'를 바이그램으로 분해
 - ⇒ '서울', '울특', '특별', '별시'
 - 분해된 하위 단어 집합에는 나뉘지지 않은 단어 자신도 포함
 - 단어 집합이 만들어지면 각 하위 단어는 고유한 벡터값을 갖게 됨
- ⇒ 하위 단어 벡터들은 단어의 벡터 표현을 구성
- ⇒ 이를 이용하여 자연어 처리 작업을 수행

6.6.2. 입력 단어 처리 방법

입력 단어	< > 더하기	N-gram 분해	전체 단어 추가
서울특별시	<서울특별시>	<서울 서울특 울특별 특별시 별시>	<서울 서울특 울특별 특별시 별시> <서울특별시>

1. 입력: '서울특별시'
2. 토큰의 양 끝에 '<' 와 '>'를 붙여 토큰의 시작과 끝을 인식할 수 있게 함
3. 분해된 토큰은 N-gram을 사용하여 하위 단어 집합으로 분해
 - 트라이그램을 사용한다면, [<서울, 서울특, 울특별, 특별시, 별시>]로 분해
4. 분해된 하위 단어 집합에는 나뉘지지 않은 토큰 자체도 포함
5. 하위 단어 집합이 만들어지면 각 하위 단어는 고유한 벡터값을 가짐
6. 각 하위 단어의 임베딩 벡터를 모두 합산하여 입력 단어의 최종 임베딩 벡터를 계산

N-gram 사용의 이점

- 다양한 N-gram을 적용해 입력 토큰을 분해하고 하위 단어 벡터를 구성
 - ⇒ 단어의 부분 문자열을 고려하는 유연하고 정확한 하위 단어 집합을 생성
 - 같은 하위 단어를 공유하는 단어끼리는 정보를 공유해 학습할 수 있음
 - ⇒ 비슷한 단어끼리는 비슷한 임베딩 벡터를 갖게 되어, 단어 간 유사도를 높일 수 있음
 - OOV 단어로 하위 단어로 나누어 임베딩을 계산할 수 있게 됨
 - 예시) '개인택시', '정보처리기사', '임대차보호법'이라는 단어가 말뭉치 내에 있을 때, 이 단어들의 하위 단어들로부터 '개인정보보호법'이라는 단어의 임베딩도 연산할 수 있음
- ⇒ 말뭉치에 등장하지 않은 단어라도 유사한 하위 단어를 가지고 있으면 유사한 임베딩 벡터를 갖게 됨

6.6.3. 모델 실습

Week11_예습과제_방민지.ipynb 참조

6.6.4. fastText의 OOV 처리

- Word2Vec 모델은 단어 사전에 존재하지 않는 단어는 임베딩을 계산할 수 없었음. 하지만 fastText는 하위 단어로 나뉘어 있기 때문에 OOV를 처리할 수 있음
- 예시
 - '사랑해요' 토큰은 '사랑', '랑해', '해요' 등의 하위 단어로 분해
 - 분해된 하위 단어의 임베딩을 모두 합해 전체 토큰의 임베딩을 계산
 - 다른 단어에서 등장했던 '사랑'이나 '해요'와 같은 하위 단어를 통해 '사랑해요'토큰의 임베딩을 계산

⇒ OOV 문제를 효과적으로 해결

- 특히 한국어 같은 많은 언어는 형태적 구조를 갖고 있기 때문에 하위 단어로 나누어 임베딩을 학습하는 접근방식을 통해 OOV 문제를 해결할 수 있음

6.7. 순환 신경망(Recurrent Neural Network, RNN)

6.7.1. 순환 신경망 개요

- 순서가 있는 **연속적인 데이터(Sequence data)**를 처리하는 데 적합한 구조를 갖고 있음
- 각 시점(Time step)의 데이터가 이전 시점의 데이터가 독립적이지 않다는 특성 때문에 효과적으로 작동

연속형 데이터

: 특정 시점 t에서의 데이터가 이전 시점($t_0, t_1, \dots, t_{n-1}$)의 영향을 받는 데이터

- 예시) 일일 주가 데이터 세트
 - 3월 7일의 주가는 3월 6일 주가의 영향을 받을 가능성이 높음
 - 3월 6일의 주가는 3월 5일 주가의 영향을 받을 가능성이 높음

연속형 데이터 ⊃ 자연어 데이터

- 자연어 데이터는 연속적인 데이터의 일종으로 볼 수 있음
 - 자연어 데이터는 문장 안에서 단어들이 순서대로 등장
 - 각 단어는 이전에 등장한 단어의 영향을 받아 해당 문장의 의미를 형성
 - 예시
 - 금요일이 지나면 _
 - 수요일이 지나면 목요일이고, 목요일이 지나면 금요일, 금요일이 지나면 _
 - 주어진 문장들에서 이전 단어들의 패턴과 의미를 고려해 다음에 올 단어를 유추
 - 첫 번째 문장에서 '금요일이 지나면'이라는 문장을 보고 '주말' 또는 '토요일' 등의 단어가 등장할 것이라고 예측할 수 있음
 - 두 번째 문장에서는 앞선 단어 패턴을 고려했을 때 '주말'보다는 '토요일'이 더 자연스러운 예측
 - 자연어는 한 단어가 이전의 단어들과 상호작용하여 문맥을 이루고 의미를 형성
 - ⇒ 자연어는 연속형 데이터의 특성을 가짐
 - ⇒ t 번째 단어는 t-1 번째의 단어에 영향을 받아 결정
 - 긴 문장일수록 앞선 단어들과 뒤따르는 단어들 사이에 강한 **상관관계(Correlation)**가 존재
 - 예시) '금요일이 지나면'이라는 문장에서 '지나면' 이라면 단어는 '금요일이'이라는 앞선 문맥과 상호 작용해 전체적인 의미를 형성
- ⇒ 자연어 처리에서는 이러한 문맥과 상호작용을 모델링해 정확한 의미 파악이 필요

6.7.2. 순환 신경망 개념

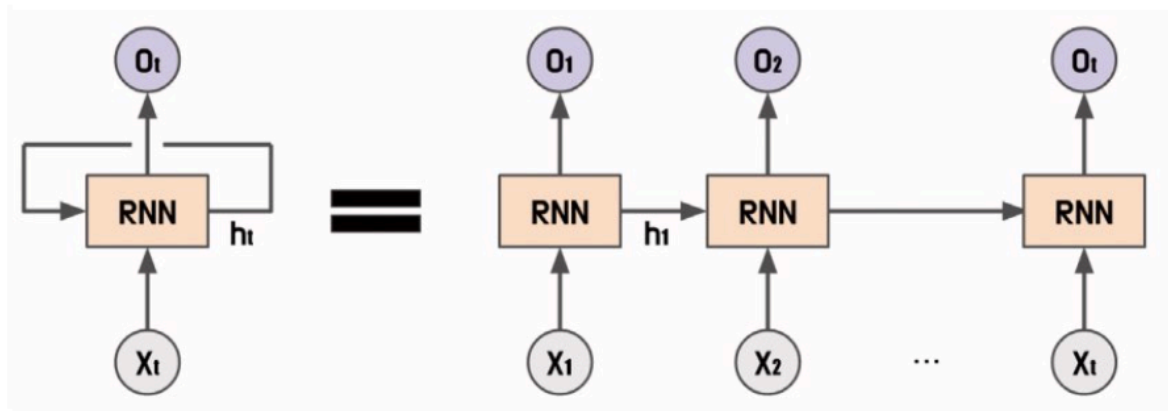
- 연속적인 데이터를 처리하기 위해 개발된 인공 신경망의 한 종류
- 이전에 처리한 데이터를 다음 단계에 활용
- 현재 입력 데이터와 함께 모델 내부에서 과거의 상태를 기억해 현재 상태를 예측하는 데 사용

사용

- 시계열 데이터, 자연어 처리, 음성 인식, 기타 시퀀스 데이터와 같은 도메인에서 널리 사용됨
 - 이러한 데이터는 일반적으로 길이가 가변적
 - 순서에 따라 의미가 있음

구조

- 연속형 데이터를 순서대로 입력받아 처리
- 각 시점마다 은닉 상태(Hidden state)의 형태로 저장
- 셀(Cell)
 - : 각 시점의 데이터를 입력으로 받아 은닉 상태와 출력값을 계산하는 노드



- 이전 시점의 은닉 상태 h_{t-1} 을 입력으로 받아 현재 시점의 은닉 상태 h_t 를 계산
- 이러한 재귀적 특성으로 인해 '순환' 신경망이라고 불림

계산 방식

- 각 시점 t에서 현재 입력값 x_t 와 이전 시점 t-1의 은닉 상태 h_{t-1} 를 이용
 - ⇒ 현재 시점의 은닉 상태 h_t 와 출력값 y_t 를 계산
- 은닉 상태의 수식

$$h_t = \sigma_h(h_{t-1}, x_t)$$

$$h_t = \sigma_h(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

- σ_h
 - 순환 신경망의 은닉 상태를 계산하기 위한 활성화 함수
 - 이전 시점 t-1의 은닉 상태 h_{t-1} 과 입력값 x_t 를 입력
 - 현재 시점의 은닉상태 h_t 를 계산
- σ_h 계산 방법
 - 가중치(W)와 편향(b)를 이용해 계산
 - W_{hh} : 이전 시점의 은닉 상태 h_{t-1} 에 대한 가중치
 - W_{xh} : 입력값 x_t 에 대한 가중치
 - b_n : 은닉 상태 h_t 에 대한 편향
- 출력값 계산 수식

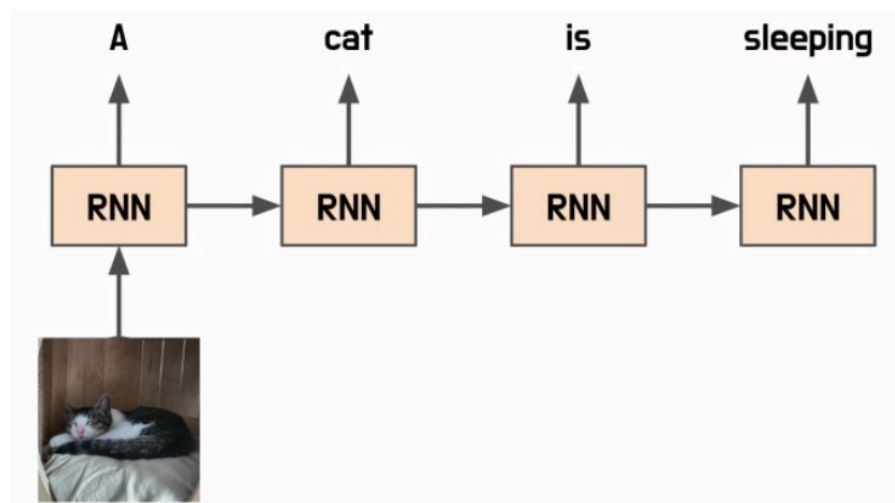
$$y_t = \sigma_y(h_t)$$

$$y_t = \sigma_y(W_{hy}h_t + b_y)$$

- σ_y
 - 출력값을 계산하기 위한 활성화 함수
 - 현재 시점의 은닉 상태 h_t 를 입력
→ 출력값 y_t 를 계산
- σ_y 계산 방법
 - 가중치(W)와 편향(b)를 이용해 계산
 - W_{hy} : 현재 시점의 은닉 상태 h_t 에 대한 가중치
 - b_y : 출력값 y_t 의 편향
 - 순환 신경망의 출력값은 이전 시점의 정보를 현재 시점에서 활용해 입력 시퀀스의 패턴을 파악하고 출력값을 예측
⇒ 연속형 데이터를 처리할 수 있음

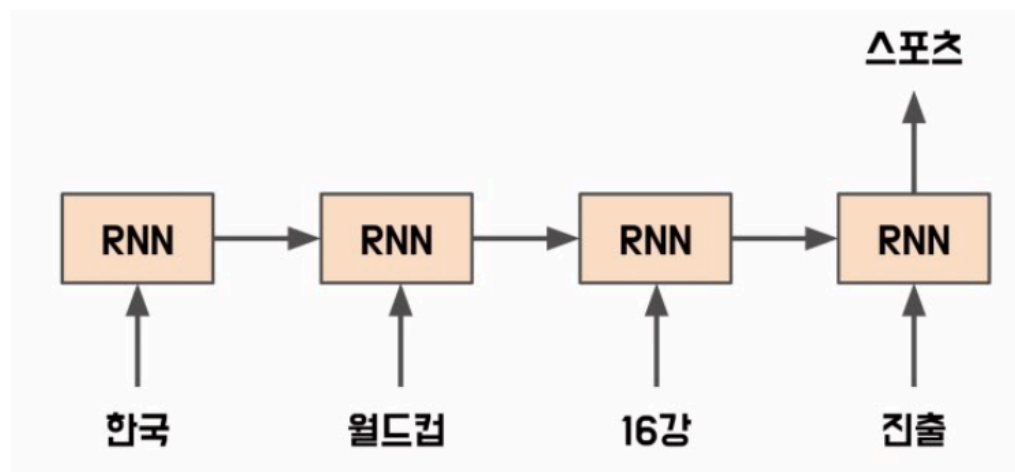
6.7.3. RNN 구조

1) 일대다 구조(One-to-Many)



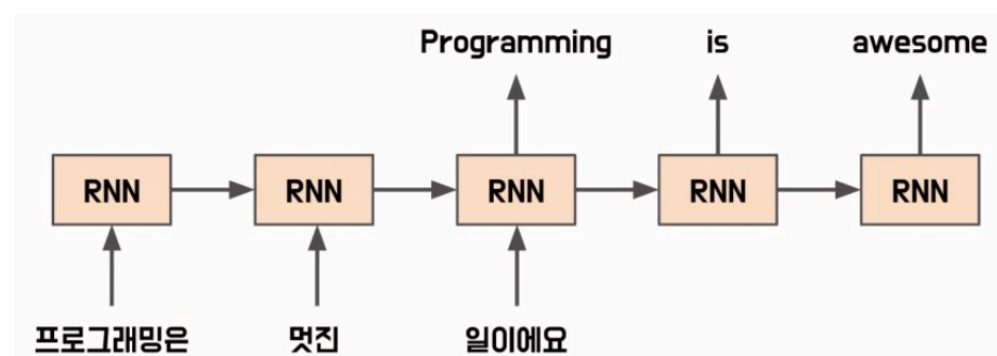
- 하나의 입력 시퀀스에 대해 여러 개의 출력값을 생성하는 구조
- 사용
 - 자연어 처리 분야
 - 일대다 구조를 사용하여 문장을 입력으로 받고, 문장에서 각 단어의 품사를 예측
 - 입력 시퀀스: 문장
 - 출력 시퀀스: 각 단어의 품사
 - 이미지 캡셔닝(Image Captioning)
 - 이미지 데이터를 입력으로 받으면 이미지에 대한 설명을 출력
 - 입력 시퀀스: 문장
 - 출력 시퀀스: 각 단어의 품사
- 일대다 구조를 구현하기 위해서는 출력 시퀀스의 길이를 미리 알고 있어야 함
⇒ 입력 시퀀스를 처리하면서 시퀀스의 정보를 활용해 출력 시퀀스의 길이를 예측하는 모델을 함께 구현해야 함

2) 다대일 구조(Many-to-One)



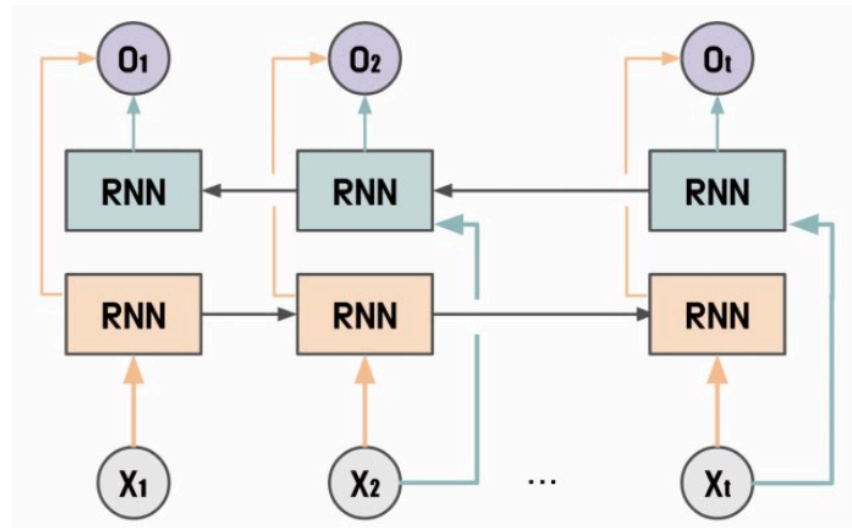
- 여러 개의 입력 시퀀스에 대해 하나의 출력값을 생성하는 구조
- 사용
 - 감성 분류(Sentiment Analysis)
 - 다대일 구조를 사용하여 특정 문장의 감정(긍정, 부정)을 예측
 - 입력 시퀀스: 문장
 - 출력값: 해당 문장의 감정
 - 문장 분류
 - 입력 시퀀스가 어떤 범주에 속하는지를 구분
 - 자연어 추론(Natural Language Inference)
 - 두 문장 간의 관계를 추론

3) 다대다 구조(Many-to-One)



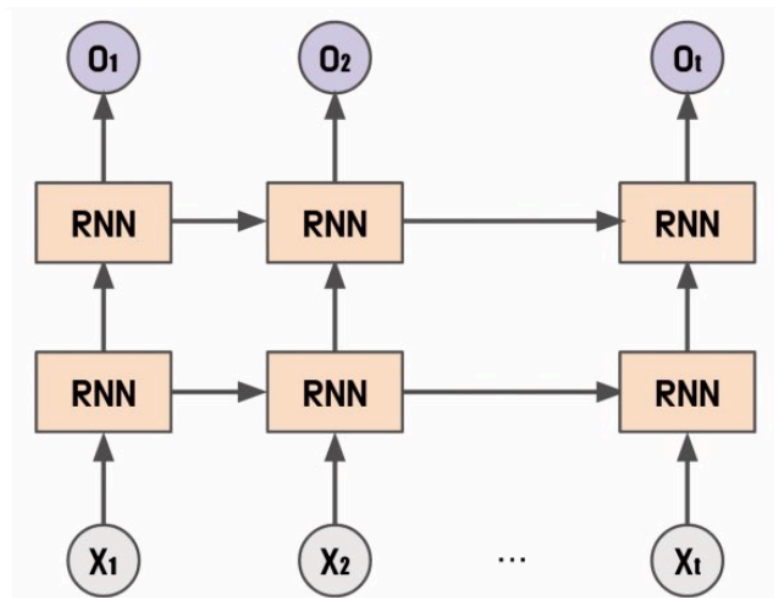
- 입력 시퀀스와 출력 시퀀스의 길이가 여러 개인 경우에 사용되는 구조
- 다양한 분야에서 활용
 - 번역기: 입력 문장에 대해 번역된 출력 문장 생성
 - 음성 인식기: 음성 신호를 입력으로 받아 문장을 출력
- 입력 시퀀스와 출력 시퀀스의 길이가 서로 다른 경우
 - 예시) 입력 문장의 길이와 출력 문장의 길이가 일치하지 않는 경우
 - 입력 시퀀스와 출력 시퀀스의 길이를 맞추기 위해 패딩을 추가하거나 잘라내는 등의 전처리 과정이 수행
- 시퀀스-시퀀스(Seq2Seq) 구조
 - 입력 시퀀스를 처리하는 인코더와 출력 시퀀스를 생성하는 디코더로 구성
 - 인코더: 입력 시퀀스를 처리해 고정 크기의 벡터를 출력
 - 디코더: 인코더에서 출력된 벡터를 입력으로 받아 출력 시퀀스를 생성

4) 양방향 순환 신경망(Bidirectional Recurrent Neural Network, BiRNN)



- 기본적인 순환 신경망에서 시간 방향을 양방향으로 처리할 수 있도록 고안된 방식
 - 기본적인 순환 신경망은 현재 시점의 입력값을 처리하기 위해 이전 시점($t-1$)의 은닉 상태를 이용
 - 양방향 순환 신경망에서는 이전 시점($t-1$)의 은닉 상태뿐만 아니라 이후 시점($t+1$)의 은닉 상태도 함께 이용
- 예시) "인생은 B와 _ 사이의 C다."
 - _에 입력될 단어를 예측해야 한다면 앞의 문장만이 아니라 뒤의 문장에도 영향을 받음
- t 시점 이후의 데이터도 t 시점의 데이터를 예측하는 데 사용될 수 있음
 - ⇒ 입력 데이터를 순방향으로 처리하는 것만이 아니라 역방향으로 거꾸로 읽어 들여 처리하는 방식
- 대부분의 연속형 데이터는 이전 시점, 이후 시점 데이터 모두와 큰 상관 관계를 갖고 있음
 - ⇒ 양방향 순환 신경망은 양방향적인 정보를 모두 고려하여 현재 시점의 출력값을 계산

5) 다중 순환 신경망(Stacked Recurrent Neural Network)



- 여러 개의 순환 신경망을 연결하여 구성한 모델
- 각 순환 신경망이 서로 다른 정보를 처리하도록 설계
- 다층 퍼셉트론과 동일한 방식
 - 단층 퍼셉트론을 여러 개 쌓아 복잡한 문제를 해결
 - 순환 신경망을 여러 층 쌓아 활용
- 여러 개 층으로 구성된 RNN은 데이터의 다양한 특징을 추출할 수 있음
 - ⇒ 성능 향상
- 층이 깊어질수록 더 복잡한 패턴을 학습할 수 있지만 학습 시간이 오래 걸리고 기울기 소실 문제가 발생할 수 있음

6.7.4. 순환 신경망 클래스

- 파이토치는 순환 신경망을 쉽게 구현할 수 있도록 순환 신경망 클래스를 제공

```

rnn = torch.nn.RNN(
    input_size, # 입력 특성 크기. 입력 특성: x
    hidden_size, # 은닉 상태 크기. 은닉 상태: h
    num_layers=1, # 계층 수: 순환 신경망의 층수. 2 이상이면 다중 순환 신경망 구성
    nonlinearity="tanh", # 순환 신경망에서 사용되는 활성화 함수의 종류 설정. tanh/relu
    bias=False, # 편향 값 사용 유무
    batch_first=True, # 배치 우선. 입력 배치 크기를 첫번째 차원으로 사용할지 여부를 설정.
    # True: [배치 크기, 시퀀스 길이, 입력 특성 크기] 형태로 입력
    # False: [시퀀스 길이, 배치 크기, 입력 특성 크기] 형태로 입력
    dropout=0, # 과적합 방지를 위한 드롭아웃 확률 설정
    bidirectional=False # 순환 신경망 구조가 양방향으로 처리할지 여부 설정
)

```

- 순전파 연산
 - 입력값(inputs)과 초기 은닉 상태(h₀)로 순방향 연산을 수행
 - 출력값(outputs)과 최종 은닉 상태(hidden)를 반환
- 입력값 차원
 - [배치 크기, 시퀀스 길이, 입력 특성 크기]로 전달
 - 초기 은닉 상태는 순방향 계층 수와 양방향 구조에 따라 전달해야 하는 크기가 달라짐
⇒ [계층 수*양방향 여부+1, 배치 크기, 은닉 상태 크기]로 구성
- 출력값, 최종 은닉 상태 반환
 - 출력값: [배치 크기, 시퀀스 길이, (양방향 여부+1)*은닉 상태 크기]
 - 최종 은닉 상태: 초기 은닉 상태와 동일한 차원으로 반환
- 순환 신경망 입출력 차원은 배치 우선 매개변수(`batch_first`)에 따라 차원의 형태가 달라지므로 매개변수 설정에 주의

6.7.5. 장단기 메모리(Long Short-Term Memory, LSTM)

장단기 메모리 개념

- 1997년 셉 호흐라이터와 유르겐 슈미트후버가 제안한 알고리즘
- 기존 순환 신경망이 갖고 있던 기억력 부족과 기울기 소실 문제를 해결한 모델

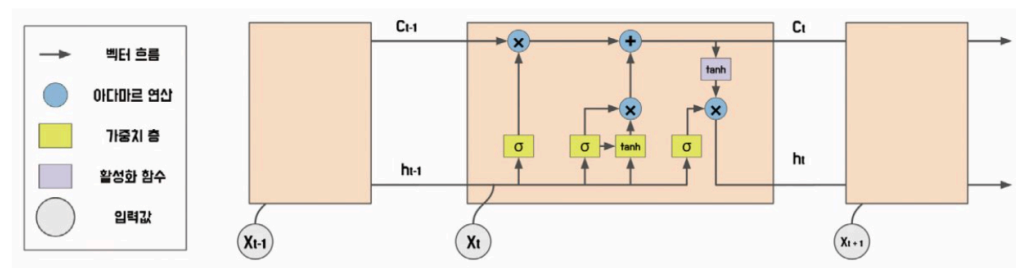
기존 순환 신경망의 단점

- 특정 시점에서 이전 입력 데이터의 정보를 이용해 출력값을 예측하는 구조
⇒ 시간적으로 먼 과거의 정보는 잘 기억하지 못함
- 어떤 연속형 데이터는 다음 시점의 데이터를 유추하기 위해 훨씬 먼 시점의 데이터에서 힌트를 얻어야 함
 - 예시) 우리는 시내에 있는 중식당에 갔다. 식당의 분위기는 괜찮았고 식사는 맛있었다. 그녀와의 대화도 즐거웠다. 우리는 함께 대화를 더 나누기 위해 _를 떠나 카페로 이동했다.
- 시간적으로 연속된 데이터를 다룰 수 있다는 장점이 있지만, 앞선 시점에서의 정보를 끊임없이 반영
⇒ 학습 데이터의 크기가 커질수록 앞서 학습한 정보가 충분히 전달되지 않음
⇒ **장기 의존성 문제(Long-term dependencies)**
- 활성화 함수로 사용되는 하이퍼볼릭 탄젠트 함수나 ReLU 함수의 특성
⇒ **역전파 과정에서 기울기 소실/기울기 폭주**

⇒ 이러한 문제를 해결하기 위해 장단기 메모리 사용

- 순환 신경망과 비슷한 구조를 가지지만, **메모리 셀(Memory cell)**과 **게이트(Gate)**라는 구조를 도입
- 장기 의존성 문제, 기울기 소실 문제 해결

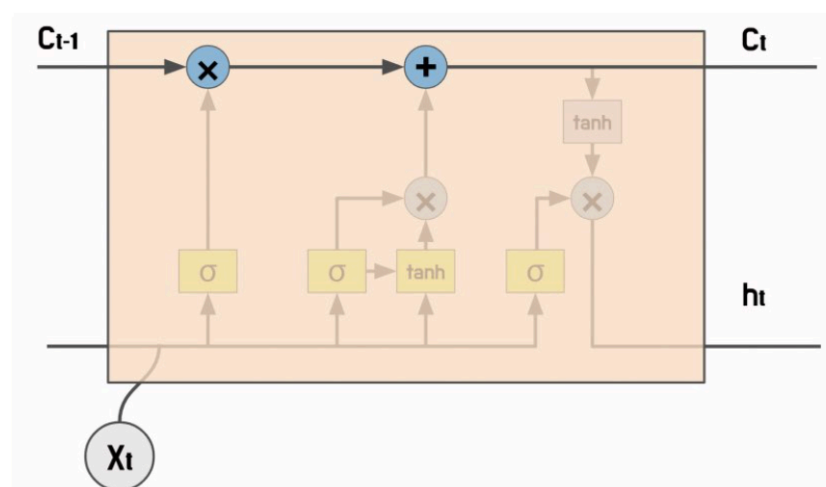
장단기 메모리 구조



- 셀상태, 망각 게이트, 기억 게이트, 출력 게이트로 정보의 흐름을 제어
- **셀 상태(Cell state)**
 - 정보를 저장하고 유지하는 메모리 역할
 - 출력 게이트, 망각 게이트에 의해 제어
- **망각 게이트(Forget gate)**
 - 이전 셀 상태에서 어떠한 정보를 삭제할지 결정하는 역할
 - 현재 입력과 이전 셀 상태를 입력으로 받아 어떤 정보를 삭제할지 결정
- **입력 게이트(Input gate)**
 - 새로운 정보를 어떤 부분에 추가할지 결정하는 역할
 - 현재 입력과 이전 셀 상태를 입력으로 받아 어떤 정보를 추가할지 결정
- **출력 게이트(Output gate)**
 - 셀 상태의 정보 중 어떤 부분을 출력할지 결정하는 역할
 - 현재 입력과 이전 셀 상태, 새로 추가된 정보를 입력으로 받아 출력할 정보를 결정

장단기 메모리 신경망 내의 셀 상태, 게이트 동작 방법

- 셀 상태의 연산 과정



- **메모리 셀**
 - 순환 신경망의 은닉 상태와 유사하게 현재 시점의 입력과 이전 시점의 은닉 상태를 기반으로 정보를 계산하고 저장하는 역할
 - 순환 신경망에서 은닉 상태: 출력값을 계산하는 데 사용
 - 장단기 메모리의 메모리 셀: 출력값 계산에 직접 사용되지 않음

구분	셀 상태 (Cell state)	은닉 상태 (Hidden state)
목적	장기 메모리 저장	현재 출력과 관련된 정보 저장
보존 능력	오래 유지됨	짧은 시간만 유지됨
흐름	거의 직접 흐름 (정보 손실 적음)	복잡한 연산과 함께 전달됨

- 장단기 메모리는 망각 게이트, 입력 게이트, 출력 게이트를 통해 어떤 정보를 버릴지, 어떤 정보를 기억할지, 어떤 정보를 출력할지 결정하는 추가적인 연산 수행
- **세 가지 게이트**
 - 모두 활성화 함수로 시그모이드를 사용
 - 시그모이드 함수는 입력값을 0~1사이의 값으로 변환

⇒ 게이트의 출력값은 각각 0~1 사이의 값으로 결정

- 망각 게이트의 출력값이 1이면 이전 시점의 기억 상태가 현재 시점의 기억 상태에 완전히 유지, 0이면 이전 시점의 기억 상태는 현재 시점의 기억 상태에 전혀 반영되지 않음

게이트 종류	역할	왜 시그모이드?
Forget Gate	이전 셀 상태에서 어떤 정보를 잊을지 결정	0~1 범위로 "이건 잊고, 이건 유지하자"를 부드럽게 판단
Input Gate	새로운 정보를 얼마나 셀 상태에 반영할지 결정	새 정보 중 어떤 걸 얼마나 기억할지 비율로 결정
Output Gate	셀 상태 중 무엇을 은닉 상태로 출력할지 결정	셀에서 꺼내는 정보의 양을 조절

1. 망각 게이트

- 입력: 현재 시점의 입력과 이전 시점의 은닉 상태
- 시그모이드 함수를 거친 값 * 메모리 셀 ⇒ 이전 시점의 메모리 셀 업데이트
- 시그모이드 함수를 거치면서 0~1 사이의 값 출력 ⇒ 어떤 정보를 유지할지, 망각할지 결정

2. 기억 게이트

- 입력: 현재 시점의 입력과 이전 시점의 은닉 상태
- 시그모이드 함수를 거친 값 * 하이퍼볼릭 탄젠트 함수를 거친 값 ⇒ 새로운 기억 값을 계산
 - 시그모이드 함수: 입력값의 중요도 결정
 - 하이퍼볼릭 탄젠트 함수: 입력값을 -1~1 사이의 값으로 변환
- 출력된 값 ⇒ 어떤 정보를 얼마나 추가할지 결정

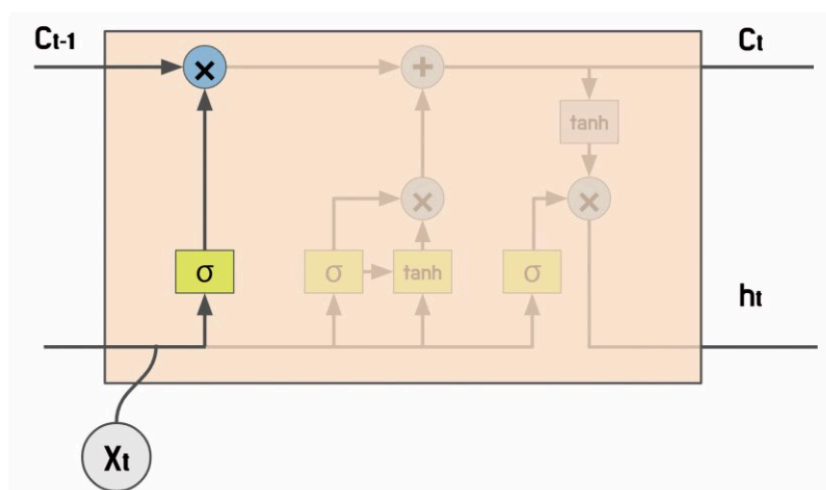
3. 출력 게이트

- 입력: 현재 시점의 입력, 이전 시점의 은닉 상태, 새로 업데이트된 메모리 셀
- 현재 시점의 출력값 계산
- 어떤 정보를 출력할지 결정

⇒ 게이트들은 입력값에 대한 가중치를 동적으로 조절하면서, 적절한 정보만을 기억하고 유지할 수 있음

장단기 메모리의 게이트 연산 과정

- 망각 게이트



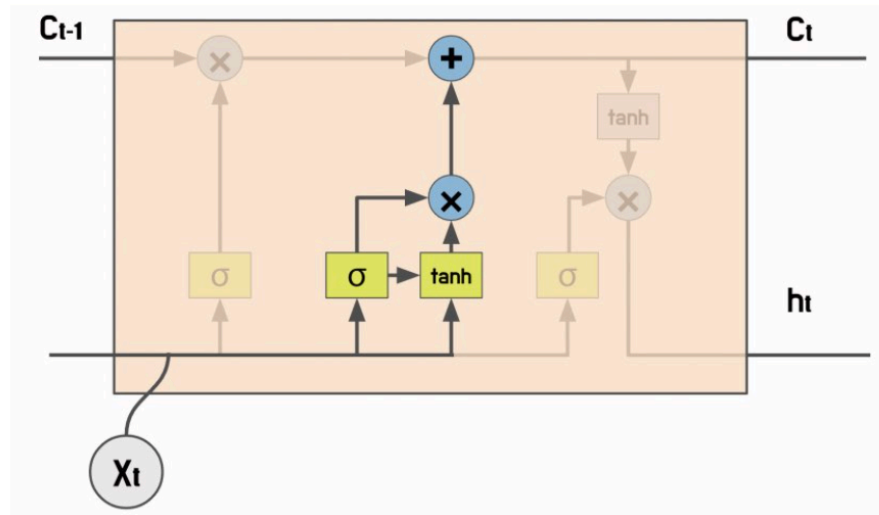
- 이전 시점의 은닉상태 h_{t-1} 과 현재 시점의 입력값 x_t 를 통해 연산을 수행
- 망각 게이트 f_t 를 계산하는 수식

$$f_t = \sigma(W_x^{(f)}x_t + W_h^{(f)}h_{t-1} + b^{(f)})$$

- 입력: 이전 시점의 메모리 셀($t-1$ 시점의 은닉 상태 h_{t-1}), 현재 시점의 입력(x_t)
- 어떤 정보를 삭제할지 결정
- 출력값 f_t : 시그모이드 활성화 함수를 사용해 계산

- $W_x^{(f)}, W_h^{(f)}$: 입력값과 은닉 상태를 위한 가중치
- $b^{(f)}$: 망각 게이트의 편향
- 두 가중치를 통해 망각 게이트 출력값을 최적화
 - ⇒ 메모리 셀을 계산하기 위한 가중치로 사용
 - ⇒ 출력값 1에 가까울수록 많은 정보를 유지
- 메모리 셀을 계산하기 위해 기억 게이트 출력값을 더해 최종 메모리 셀 값을 계산

• 기억 게이트



- 기억 게이트 계산하는 수식

$$g_i = \tanh(W_x^{(g)}x_t + W_h^{(g)}h_{t-1} + b^{(g)})$$

$$i_i = \text{sigmoid}(W_x^{(i)}x_t + W_h^{(i)}h_{t-1} + b^{(i)})$$

- g_i 와 i_i 로 구성, 망각 게이트와 동일한 연산으로 계산
 - g_i : 활성화 함수로 하이퍼볼릭 탄젠트 사용
 - i_i : 활성화 함수로 시그모이드 사용
- g_i
 - -1~1 값을 가짐
 - 이전 시점의 은닉 상태와 현재 시점의 입력값은 모두 [-1, 1] 범위 안에 존재
 - g_i 만으로는 새로운 은닉 상태를 계산하기 위해 이 은닉 상태를 얼마나 기억할지 제어하기가 어려움
 - ⇒ i_i 값으로 새로운 은닉 상태의 기억을 제어
- i_i
 - 0~1 값을 가짐
 - 현재 시점에서 얼마나 많은 정보를 기억할 것인지를 결정하는 가중치 역할을 함
 - 1에 가까울수록 기억할 정보가 많아지고, 0에 가까울수록 정보를 망각

• 메모리 셀

- 망각 게이트와 기억 게이트 정보로 현재 시점의 메모리 셀 값을 계산
- 최종 메모리 셀 계산 수식

$$c_t = f_t \odot c_{t-1} + g_i \odot i_t$$

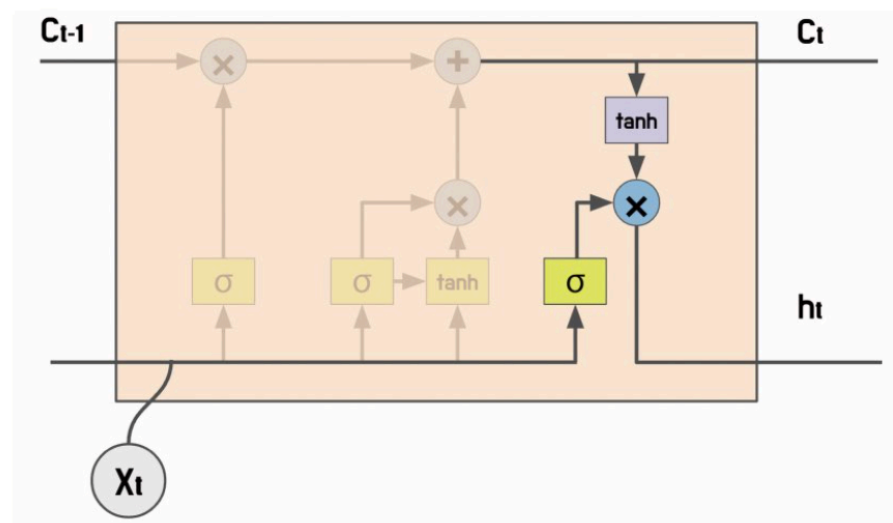
- f_t : 망각 게이트 출력값
- c_{t-1} : 이전 시점의 메모리 셀 값

- $\odot$ (아다마르 곱): 원소별 곱셈 연산
- 망각 게이트 값 f_t 가 0에 가까울수록 이전 시점의 메모리 셀 값이 현재 시점의 메모리 셀 값에 영향을 미치지 않게 됨
- g_i, i_i : 기억 게이트 출력 값
- 기억 게이트도 아다마르 곱을 통해 계산, 망각 게이트와 합산

⇒ 망각 게이트는 이전 시점의 메모리 셀을 얼마나 유지할지 결정, 기억 게이트는 현재 시점의 새로운 정보를 얼마나 받아들일지를 결정

- 메모리 셀 계산 후 어떤 정보를 출력할지 결정해야 함

• 출력 게이트



- 현재 시점의 은닉 상태를 제어
- 출력 게이트 계산하는 수식

$$o_t = \sigma(W_x^{(o)}x_t + W_h^{(o)}h_{t-1} + b^{(o)})$$

- 망각 게이트에서 사용된 수식과 동일
- 활성화 함수: 시그모이드
 - 입력값을 $[0, 1]$ 범위로 변환
 - 현재 시점의 은닉 상태 값을 얼마나 사용할지 결정
- 은닉 상태 갱신하는 수식

$$h_t = o_t \odot \tanh(c_t)$$

- 출력 게이트와 현재 메모리 셀을 연산하여 새로운 은닉 상태를 계산
- 출력 게이트와 하이퍼볼릭 탄젠트를 적용한 메모리 셀 값으로 계산
 - 출력 게이트: $[0, 1]$ 범위
 - 하이퍼볼릭 탄젠트가 적용된 메모리 셀: $[-1, 1]$ 범위
 - 두 값을 아다마르 곱 연산
 - ⇒ 현재 시점의 은닉 상태가 이전 시점의 은닉 상태에서 얼마나 영향을 받는지 계산할 수 있게 됨

6.7.6. 장단기 메모리 클래스

- 파이토치에서 쉽게 구현할 수 있도록 장단기 메모리 클래스를 제공
- 순환 신경망 클래스와 거의 유사한 구조

```
lstm = torch.nn.LSTM(
    input_size,
```

```
hidden_size,
num_layers=1,
bias=False,
batch_first=True,
dropout=0,
bidirectional=False,
proj_size=0
)
```

- 활성화 함수를 명확하게 정의해 사용
⇒ 순환 신경망에서 사용하는 활성화 함수(nonlinearity) 매개 변수는 사용하지 않음
- 투사 크기(`proj_size`) 매개 변수 사용
 - 장단기 메모리 계층의 출력에 대한 선형 투사(Linear Projection) 크기를 결정
 - 0보다 큰 경우
 - 은닉 상태를 선형 투사를 통해 다른 차원으로 매핑
 - 출력 차원을 줄이거나 다른 차원으로 변환할 수 있음
 - 0인 경우
 - 은닉 상태의 차원을 변환하지 않고 유지
 - LSTM은 출력 차원을 조절할 때 투사 크기 매개변수로 모델의 크기와 복잡도를 변경할 수 있음
⇒ 투사 크기는 은닉 상태 크기(`hidden_size`)보다 작은 값으로 설정
- 순전파 연산
 - 입력값(inputs), 초기 은닉 상태(h_0), 초기 메모리 셀 상태(c_0)로 순방향 연산 수행
 - **입력값 차원**: 순환 신경망 클래스에서 사용하던 입력값 차원 형태와 동일한 형태. [배치 크기, 시퀀스 길이, 입력 특성 크기]를 전달
 - **초기 은닉 상태**: 순환 신경망 클래스와 동일한 구조. 선형 투사를 통해 출력층의 차원을 변경할 수 있으므로 투사 크기가 0보다 큰 경우 출력층의 차원을 투사 크기로 입력
 - **초기 메모리 셀**: 순환 신경망 클래스의 초기 은닉 상태와 동일한 구조. [계층수*양방향 여부+1, 배치 크기, 은닉 상태 크기]. 선형 투사를 적용하더라도 은닉 상태 크기를 사용
 - 출력값(outputs), 최종 은닉 상태(h_n), 최종 메모리 셀 상태(c_n) 반환
 - 출력값: 순환 신경망과 동일한 수식 사용. 선형 투사가 적용 됐으므로 [배치 크기, 시퀀스 길이, (양방향 여부+1)*투사크기(또는 은닉 상태 크기)]로 구성
 - 최종 은닉 상태: 초기 은닉 상태와 동일한 차원
 - 순환 신경망 입출력 차원은 배치 우선 매개변수에 따라 차원의 형태가 달라지므로 매개변수 설정에 주의
- 순환 신경망과 비슷한 구조를 갖지만, 투사 크기가 적용되는 경우 차원 계산 방식이 달라짐
- 메모리 셀의 상태도 입력으로 전달해야 하며, 은닉 상태와 메모리 셀 상태를 튜플로 묶어야하므로 사용에 주의

6.7.7. 모델 실습

6.8. 합성곱 신경망(Convolutional Neural Network, CNN)

6.8.1. 합성곱 신경망 개념

- 컴퓨터 비전 분야의 데이터를 분석하기 위해 사용되는 인공 신경망의 한 종류
- 합성곱 신경망은 입력 데이터의 지역적인 특징을 추출하는 데 특화된 구조를 갖고 있음
⇒ **합성곱(Convolution) 연산** 사용

합성곱 연산

- 이미지의 특정 영역에서 입력값의 분포/변화량을 계산해 출력 노드를 생성
- 특정 영역 안에서 연산을 수행
 - ⇒ 지역 특징(Local Features)을 효과적으로 추출
- 이미지 데이터는 고정된 프레임 내에 객체들의 위치와 형태가 자유분방
 - ⇒ 여러 영역의 지역 특징을 조합
 - ⇒ 입력 데이터의 전반적인 전역 특징(Global Features)를 파악

합성곱 신경망과 자연어 처리

- 자연어 처리에도 우수한 성능을 보임
- 순환 신경망의 제약
 - 연산 순서의 제약으로 병렬 처리가 어려움
 - 입력 데이터의 길이가 길어질수록 처리 속도가 느려짐
 - 많은 수의 가중치를 사용하면 학습이 어려워져 깊은 신경망을 구성하기 어려움
- 2014년 사전 학습된 임베딩과 합성곱 신경망을 이용한 문장 분류 모델의 등장
 - ⇒ 자연어 처리에서도 CNN 사용
 - 입력 데이터의 길이에 상관없이 병렬 처리가 가능
 - 학습에 필요한 가중치 수를 줄여 깊은 신경망 구성 가능

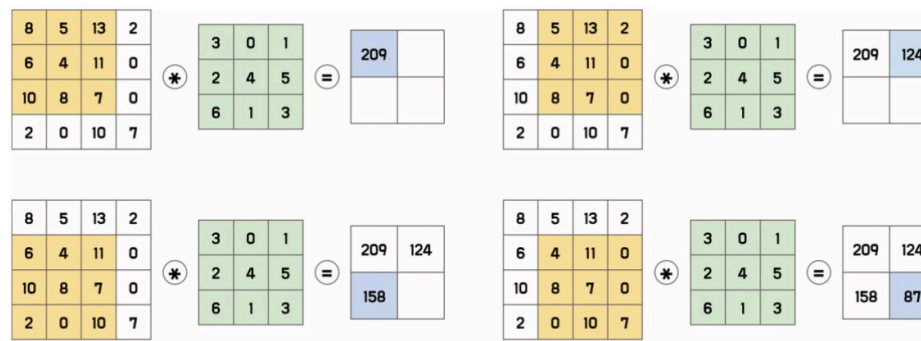
6.8.2. 합성곱 계층 구조

합성곱 계층

- 입력 데이터와 필터를 합성곱해 출력 데이터를 생성하는 계층
- 이미지, 음성 데이터와 같은 고차원 데이터를 처리하는 데 주로 사용
- 필터를 사용해 데이터의 특징을 추출
 - ⇒ 데이터의 지역적인 패턴을 인식
- 입력 데이터의 모든 위치에서 동일한 필터를 사용
 - ⇒ 모델 매개변수를 공유
 - ⇒ 모델이 학습해야 할 매개변수 수가 감소
 - ⇒ 과적합 방지
- 입력 데이터에서 특징을 추출할 때, 해당 특징이 이미지 내 다른 위치에 존재하더라도 필터를 사용해 특징을 추출
 - ⇒ 특징이 어디에 있어도 동일하게 추출
- 합성곱 계층 여러 겹을 쌓아 모델을 구성
- 계층이 많아질수록 모델의 복잡도 증가, 다양한 특징을 추출해 학습

필터/커널/원도

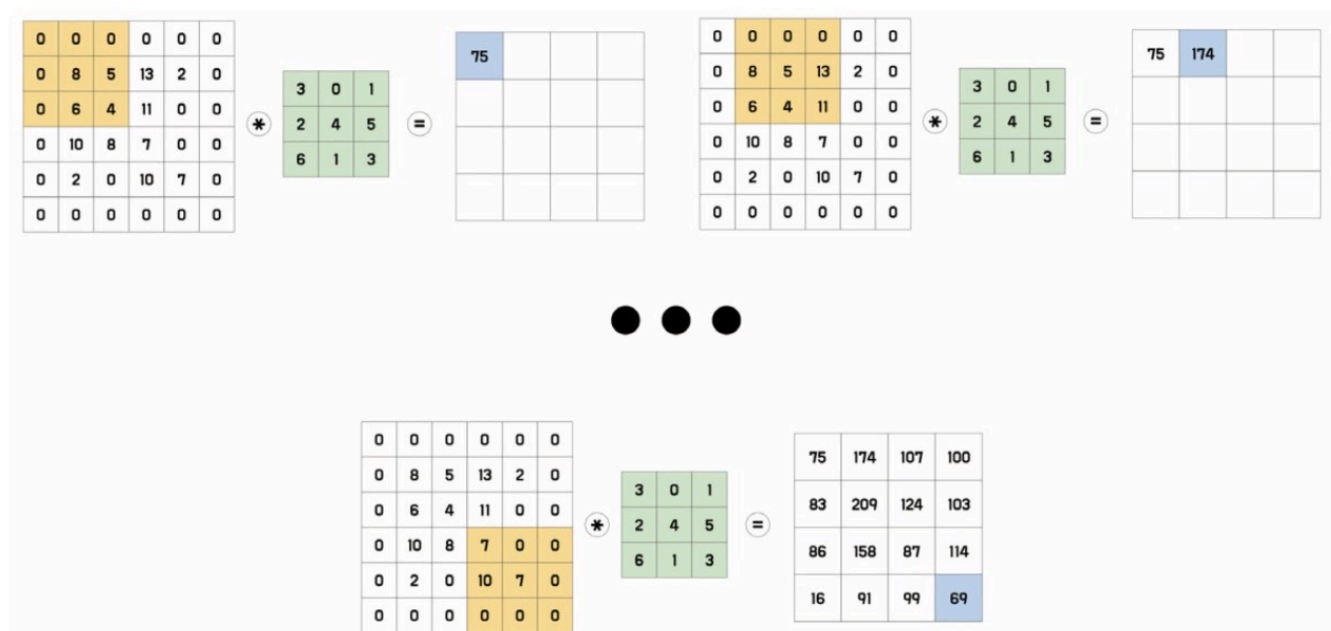
- 3×3, 5×5 같은 작은 크기의 정방형으로 구성
- 일정 간격으로 이동하면서 입력 데이터와 합성곱 연산을 수행해 특징 맵을 생성
- 필터 영역마다 합성곱 연산이 수행
- 필터의 가중치가 모델 학습 과정에서 갱신
- 여러 개의 필터를 사용하여 다양한 특징을 추출
 - ⇒ 입력 이미지의 다양한 특징을 인식, 분류
- 예시) 4x4 크기의 이미지에 3x3 크기의 필터를 1 간격으로 이동하며 특징 맵 추출



- 2×2 특징 맵 추출
- 4×4 입력 이미지의 각 픽셀 값에 3×3 필터를 1 간격으로 이동시키며 합성곱 연산을 수행
- 입력값: 이미지의 픽셀값
- 필터의 값: 학습을 통해 변경되는 가중치. 임의의 초깃값 설정
- 입력 이미지에서 필터와 대응하는 부분을 곱한 후 모두 더한 값을 특징 맵의 한 원소로 사용
- 필터는 입력 이미지 왼쪽 상단 → 오른쪽 하단으로 이동하며 합성곱 연산을 반복하며 특징 맵 전체를 계산
- 특징 맵은 다음 합성곱 계층의 입력으로 사용되어 동일한 합성곱 연산 과정을 반복
- 각 계층에서 하나의 필터가 여러 번 사용, 이를 공유 가중치로 공유
⇒ 이미지 내에서 어느 위치에서도 동일한 패턴을 학습

패딩

- 합성곱 연산을 수행하면 출력값인 특징 맵의 크기가 작아짐
⇒ 제약이 생김
 - 합성곱 신경망을 더 깊게 쌓는 데 제약사항이 될 수 있음
 - 성능에 악영향을 끼칠 수 있음
 - 이미지의 가장자리 정보는 다른 위치에 있는 정보에 비해 학습하기가 어려움
- 이러한 제약을 해결하기 위해 입력 이미지/입력으로 사용되는 특징 맵 가장자리에 특정 값을 덧붙이는 패딩을 추가
- 가장자리에 덧붙이는 패딩 값은 0으로 할당 ⇒ 제로 패딩
- 예시) 4×4 크기의 이미지를 제로 패딩한 뒤, 3×3 필터를 통해 4×4 크기의 특징 맵을 계산하는 방법



- 출력 값의 크기가 작아지지 않고 입력값 크기와 동일한 크기로 특징 맵이 계산
- 패딩을 사용해 입력 데이터의 크기 조정, 작은 크기의 필터로도 이미지 전체에 대한 정보를 반영

간격

- 필터가 한 번에 움직이는 크기
- 출력 데이터의 크기를 조절
- 간격을 조정하여 입력 데이터의 공간적인 정보를 유지/감소 할 수 있음

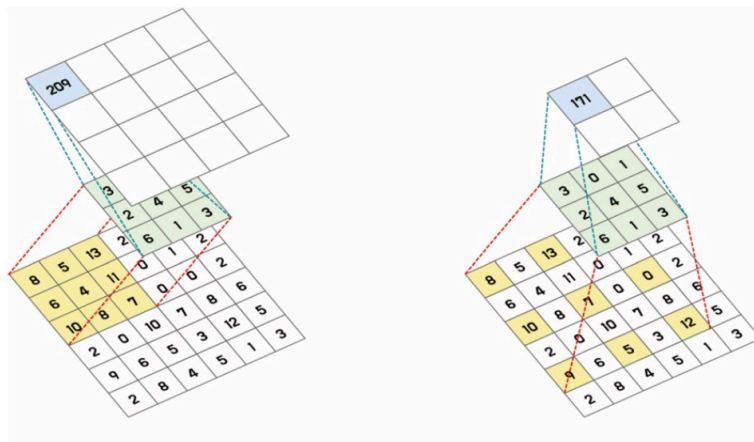
- 입력 데이터의 공간적인 정보: 픽셀 간의 상대적인 위치/거리에 대한 정보
 - 간격 작으면 입력 데이터의 공간적인 정보 보존
 - 간격 크면 입력 데이터의 공간적인 정보 감소
- 간격을 조절해 학습해야 하는 모델 매개변수의 수 감소
 - ⇒ 모델의 복잡도를 낮추고 과대적합을 방지

채널

- 입력 데이터와 필터 간의 연산이 수행되는 곳
- 3차원(높이×너비×채널) 구조에서 동일한 위치의 값들끼리 연산이 이뤄짐
 - ⇒ 입력 데이터의 공간 정보를 유지하면서 추출되는 특징을 확장
 - 각 채널은 입력의 서로 다른 측면(예: RGB 이미지의 R, G, B)을 독립적으로 처리
 - 채널별로 공간적 특징을 추출해, 원래의 픽셀 위치·거리 정보를 보존
- 채널 개수
 - 합성곱 계층에서 설정
 - 모델 구조/목적에 따라 달라짐
 - 채널 개수가 많아질수록 학습할 수 있는 특징의 다양성이 증가
 - ⇒ 표현력 높아짐
- 출력 채널 많은 경우
 - 각 채널은 입력 데이터에서 서로 다른 특징을 학습
 - 모델은 더 많은 종류의 특징을 학습
 - ⇒ 더 복잡한 문제 해결
 - 학습시간, 메모리 사용량 증가
 - ⇒ 과적합 위험

팽창(Dilation)

- 합성곱 연산을 수행할 때 입력 데이터에 더 넓은 범위의 영역을 고려할 수 있게 하는 기법
- 필터와 입력 데이터 사이에 간격을 두는 방법
- 일반적으로 합성곱 계층에서 팽창값은 1로 사용해 한칸만 띄워 사용
- 팽창 값이 커질수록 필터가 입력 데이터를 바라보는 범위가 넓어짐



- 필터의 크기를 키우지 않고 입력 데이터에 더 넓은 영역을 고려할 수 있게 함
 - ⇒ 더 깊고 복잡한 모델을 구성할 수 있음
- 메모리 사용량을 줄일 수 있음
- 입력 데이터의 범위가 커져 오히려 연산량이 늘어날 수 있음
- 팽창 크기가 너무 크면 인접한 픽셀값 고려 X
 - ⇒ 공간적 정보 보존 X

⇒ 특징 추출의 효과가 떨어질 수 있음

6.8.3. 합성곱 계층 클래스

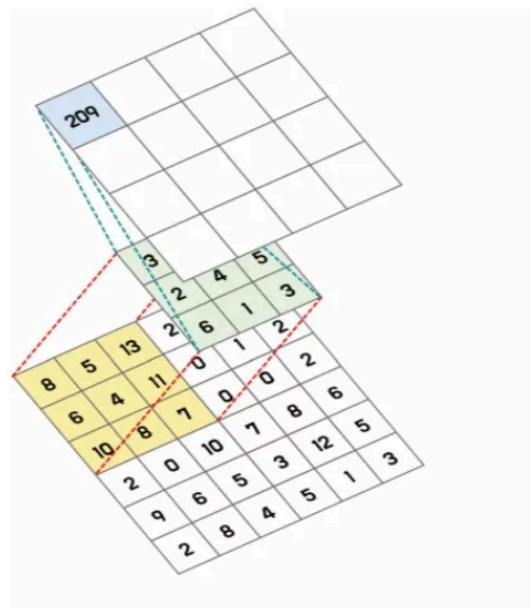
- 파이토치에서 합성곱 계층 클래스를 사용해 간단하게 신경망 구현
- 2D 합성곱 계층 클래스

```
conv = torch.nn.Conv2d(  
    in_channels, # 입력 데이터 채널 크기  
    out_channels, # 출력 데이터 채널 크기  
    kernel_size, # 합성곱 연산의 필터 크기  
    stride=1,  
    padding=0,  
    dilation=1,  
    groups=1,  
    # 입력 채널과 출력 채널을 하나의 그룹으로 묶는 것.  
    # 2 이상 => 채널을 더 작은 그룹으로 나눠 각각 합성곱 연산 수행. 입력 채널과 출력 채널의수가 그룹 수로 나누어 떨어져야 함  
    # 합성곱 연산 시 모델의 매개변수 수를 줄일 수 있음  
    bias=True, # 계층에 편향 값 포함 여부  
    padding_mode="zeros" # 패딩 영역에 할당되는 값(zeros/reflect/replicate)  
)
```

- 출력 크기 수식
 - 필터를 통해 입력 데이터의 공간적인 정보를 유지/감소
⇒ 출력 크기를 합성곱 계층에서 제어

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times padding - dilation \times (kernel_size - 1) - 1}{stride} + 1 \right\rfloor$$

- 합성곱 계층을 통과하는 입력값의 크기 수식(L_{in})는 같은 형태로 출력 크기 계산



$$L_{in} = 6, kernel_size = 3, stride = 1, padding = 0, dilation = 1$$
$$L_{out} = \left\lfloor \frac{6 + 2 \times 0 - 1 \times (3 - 1) - 1}{1} + 1 \right\rfloor = 4$$

- 값에 따라 소수점 발생 ⇒ 버림 연산

6.8.4. 활성화 맵(Activation Map)

- 합성곱 계층의 특징 맵에 활성화 함수를 적용해 얻어진 출력값

- 합성곱 계층에서 합성곱 연산을 통해 특징 맵 추출 → 비선형성을 추가하기 위해 활성화 함수 적용(ReLU) → 다음 계층의 입력값으로 사용
- 합성곱 연산, 활성화 함수를 여러 번 반복하여 신경망 구성
⇒ 입력 이미지에서 추출된 추상적인 특징 학습

활성화 함수의 필요성

- 합성곱 계층의 출력값에 활성화 함수를 적용하지 않으면 결국값이 그대로 다음 계층으로 전달
⇒ 모델이 선형적인 결합만 수행
⇒ 복잡한 패턴, 추상적인 특징을 학습하는 것이 어려워짐
- 활성화 함수를 적용
⇒ 모델이 비선형성을 가지게 됨
⇒ 입력 데이터에서 다양한 추상적인 특징을 학습

활성화 맵의 필요성

- 합성곱 모델을 분석하는 데 매우 중요한 정보를 제공
- 활성화 맵 시각화
 - 모델이 어떤 특징 학습하는지
 - 어떤 부분 활성화되어 있는지 이해

6.8.5. 풀링

- 특징 맵의 크기를 줄이는 연산
- 합성곱 계층 다음에 적용
- 특징 맵의 크기를 줄여 연산량 감소, 입력 데이터의 정보를 압축

풀링 작동 방식

- 일정한 크기의 필터 내 특정 값 선택
- 특정 값 선택 방식
 - 최댓값 풀링(Max Pooling)
 - 필터 내 원소값 중 가장 큰 값을 선택 → 특징 맵 크기 감소
 - 평균값 풀링(Average Pooling)
 - 필터 내 원소값의 평균값 → 특징 맵 크기 감소

풀링의 효과

- 입력 데이터의 공간적 크기 줄임
⇒ 계산 비용 감소
- 인근 영역에 대한 연산 적용
⇒ 입력 데이터의 특징 위치가 변경되더라도 공간적 정보 유지

풀링 단점

- 입력 데이터의 위치 정보를 일부 손실
⇒ 세밀한 위치 정보가 필요한 작업에서는 성능 저하
⇒ 풀링보다는 합성곱 계층의 간격을 설정해 공간적 크기를 줄이는 방법 사용

풀링 클래스

- 파이토치에서 풀링 클래스로 간단히 풀링 적용
- 2차원 최댓값 풀링 클래스


```
pool = torch.nn.MaxPool2d(
    kernel_size,
    stride=None,
    padding=0, # 패딩모드 적용 X
    dilation=1
)
```

- 출력 크기 수식
 - 합성곱 계층 출력 수식과 동일

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times padding - dilation \times (kernel_size - 1) - 1}{stride} + 1 \right\rfloor$$

- 2차원 평균값 풀링 클래스

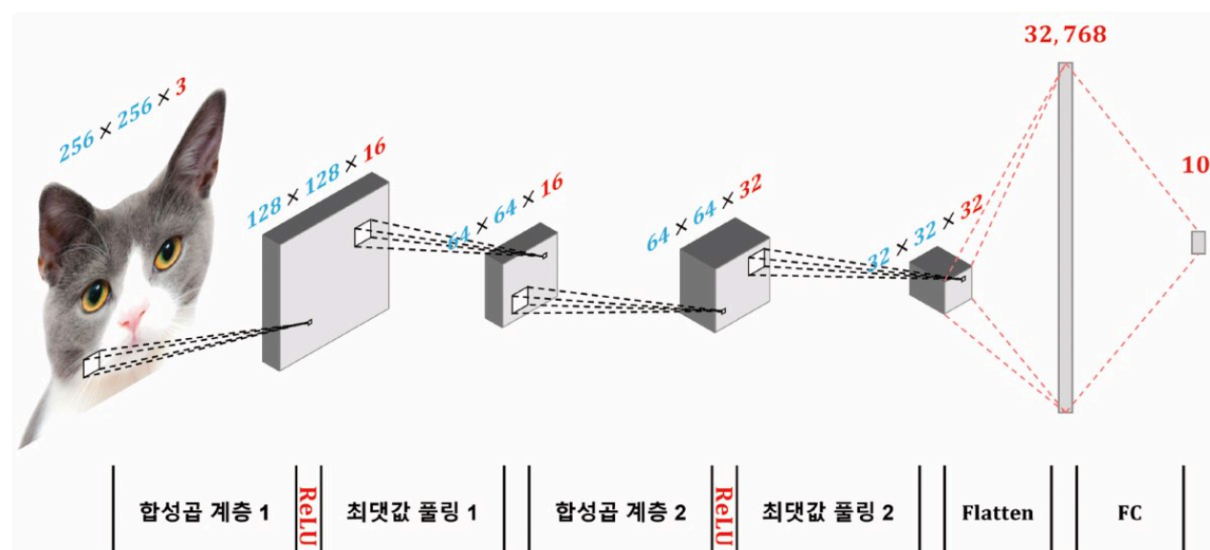
```
pool = torch.nn.AvgPool2d(
    kernel_size,
    stride=None,
    padding=0,
    count_include_pad=True # 패딩 포함: 패딩 영역의 값을 평균 계산에 포함할지 여부
    # 팽창 지원 X
)
```

- 출력 크기 수식
 - 팽창 값을 1이라 가정하면 최댓값 풀링의 출력 수식과 동일

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times padding - kernel_size}{stride} + 1 \right\rfloor$$

6.8.6. 완전 연결 계층(Fully Connected Layer, FC)

- 각 입력 노드가 모든 출력 노드와 연결된 상태
 - ⇒ 입력과 출력 간의 모든 가능한 관계를 학습
- 출력 노드의 수를 조절
 - ⇒ 모델의 복잡성, 용량을 조절하는 데 사용
- 합성곱 신경망 입력: 특징맵
 - 입력된 3차원 특징맵 → 평탄화 작업 → 1차원 벡터로 변경 → 완전 연결 계층의 가중치와 내적 연산 → 출력값 계산



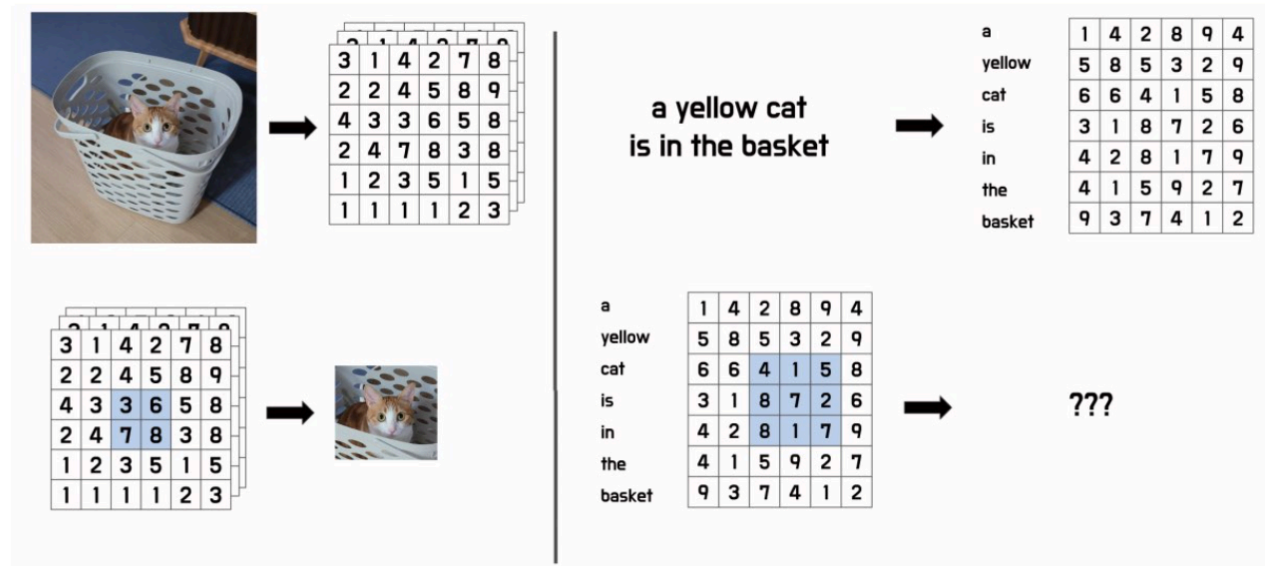
- 특성 맵의 공간 정보를 보존하기 위해 합성곱 계층으로 구성된 네트워크를 구성 → 완전 연결 계층을 추가하여 고수준의 작업 수행

- 이전 계층에서 추출된 특징을 활용하여 최종적인 분류 작업 수행
 - 소프트맥스, 시그모이드 같은 활성화 함수를 적용해 분류 모델 구성
- ⇒ 최종 출력 결정

6.8.7. 1차원 합성곱

텍스트 데이터와 CNN

- 텍스트 데이터의 임베딩 값은 입력 순서를 제외하면 입력값의 위치가 의미를 가지지 않음
- 텍스트 데이터에 이미지 데이터와 같이 2차원 합성곱 필터를 사용하면 텍스트 정보를 제대로 학습할 수 없음



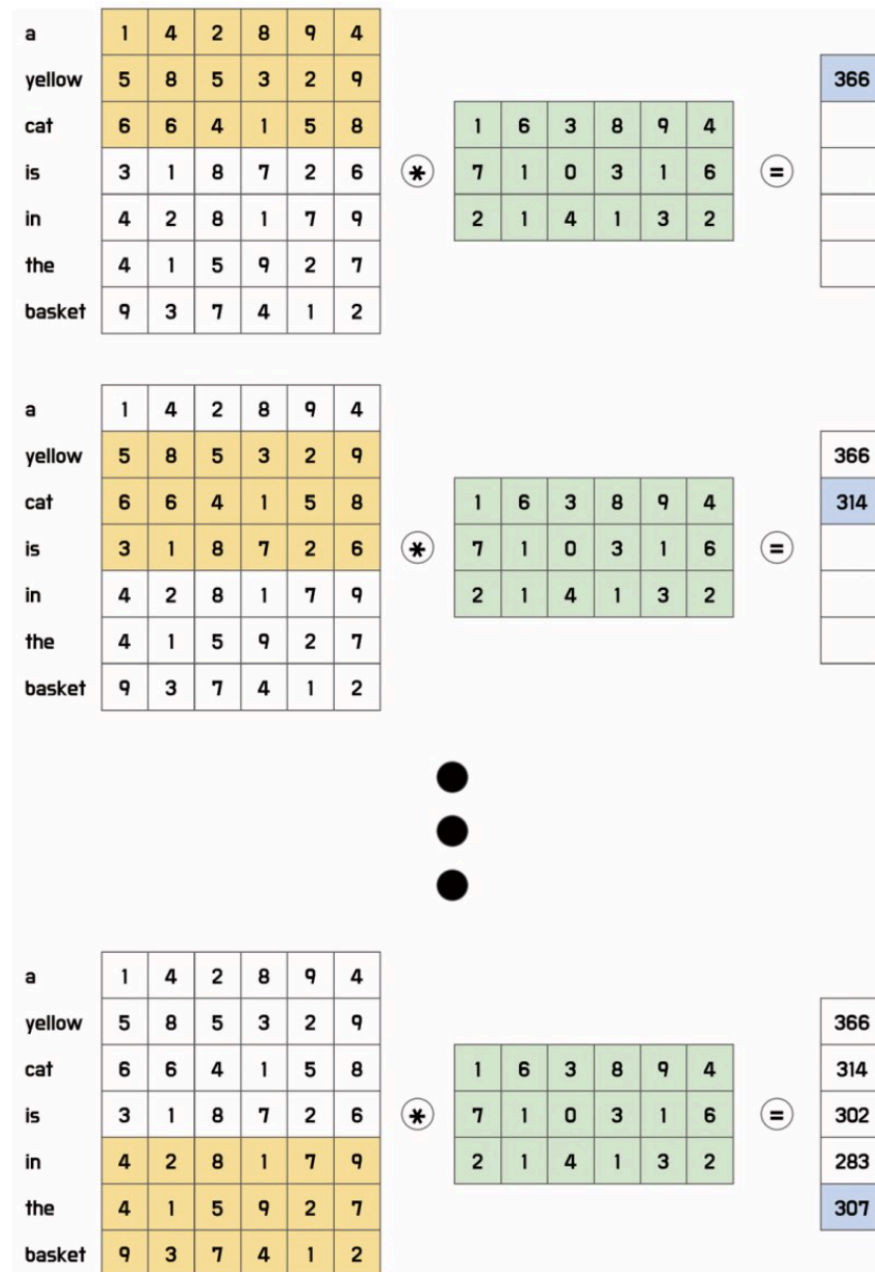
- 텍스트 데이터에는 1차원 합성곱 적용

1차원 합성곱

- 입력 데이터가 1차원 벡터인 경우에 대한 합성곱 연산을 수행
- 텍스트 처리에서 많이 사용
 - 문장을 단어 단위로 분리하여 각 단어를 임베딩하여 나온 1차원 벡터 데이터를 입력값으로 사용
- 수직 방향으로만 이동하는 필터를 적용
- 필터 크기: 필터의 높이에만 영향
- 필터 너비: 입력 임베딩 크기

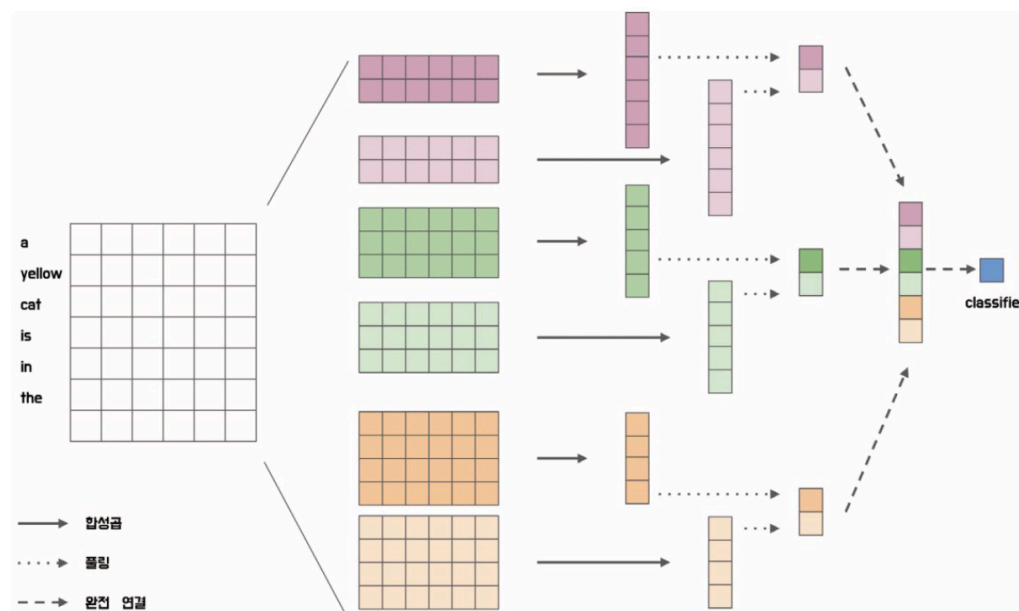
1차원 합성곱 수행

- 예시) 텍스트 임베딩 입력값에 크기가 3인 1차원 합성곱 연산을 1씩 이동하며 수행



- 필터 크기가 3인 합성곱 임베딩 수행 ⇒ 인접한 3개의 토큰에 대해 연산 수행
- N-gram과 유사한 개념
- 텍스트 순서에 따른 정보 학습

1차원 합성곱을 이용한 출력 벡터 계산



- 다양한 크기의 합성곱을 사용하여 여러 종류의 정보를 추출
- 1차원 합성곱 수행 ⇒ 1차원 벡터를 출력값으로 얻음
⇒ 풀링을 적용하면 하나의 스칼라 값이 도출
- 크기가 다른 여러 개의 합성곱 필터를 사용하면 여러 개의 스칼라값을 얻을 수 있음
⇒ 다양한 크기의 출력 벡터를 모아 하나의 벡터로 연결하여 하나의 **특징 벡터**로 만들 수 있음
- 특징 벡터를 이용해 분류, 예측 등의 작업 수행

6.8.8. 모델 실습

Week11_예습과제_방민지.ipynb 참조